POSTGRESQL ON KUBERNETES SUMMIT · PGCONF.EU 2026, VALENCIA

The Anatomy of a PostgreSQL Backup CRD

pgBackRest, Multi-Repo, and Tiered Storage

George Kechagias, Senior Software Engineer, Percona

Percona Operator for PostgreSQL

2026.pgconf.eu/community-events/pg_on_kubernetes/



PERCONA®

`kubectl apply -f backup.yaml`

What sits between that and a backup in object storage

- The **engine**: pgBackRest, and how it hooks into PostgreSQL
- The **CRD**: what the operator exposes, and what it hides
- The **storage**: multi-repo, and the tiering it unlocks

> One thing drives all three: the **Percona Operator for PostgreSQL**. It writes pgBackRest's config, runs every command it issues, and reconciles the result back into Kubernetes objects. pgBackRest does the work; the operator decides when and with what.

THE ENGINE

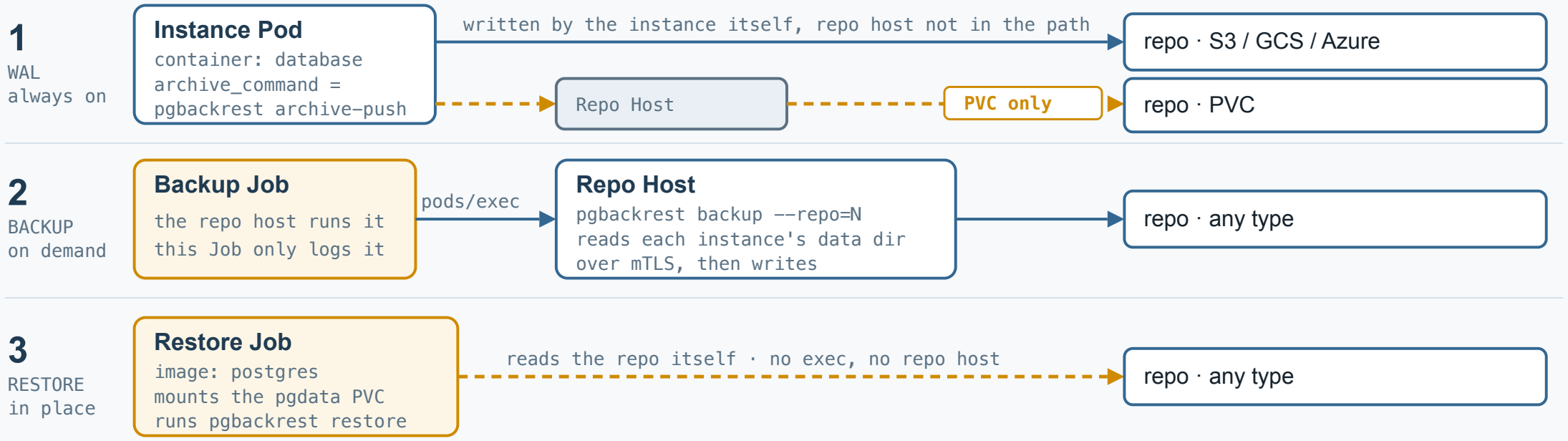
pgBackRest

The vocabulary

- **Stanza**: one PostgreSQL cluster's config. The operator always names it `db`
- **Repo**: one backup destination. `repo1` … `repo4`
- **Backup**: `full`, `differential` (since last full), `incremental` (since last backup of any type)
- **WAL archive**: the continuous stream that makes PITR possible

> Everything the operator does is `pgbackrest backup`, `restore`, `archive-push`, `expire`, with the config generated for you.

How pgBackRest attaches to PostgreSQL



Three paths, three different actors, one stanza. **WAL** is pushed by the instance itself. A **backup** is a Job that execs into the repo host and relays its output. A **restore** is a Job that reads the repo directly and never touches the repo host.

THE CRD

It all hangs off repos[ ]

```yaml
backups:
  pgbackrest:
    global:
      repo1-retention-full: "2"          # retention lives HERE, not on the repo
    repos:
      - name: repo1                      # ^repo[1-4]$
        volume: { volumeClaimSpec: { resources: { requests: { storage: 100Gi } } } }
        schedules: { full: "0 2 * * 0", incremental: "0 2 * * 1-6" }
      - name: repo2
        s3: { bucket: pg-backups, region: eu-west-1 }
        schedules: { full: "0 4 * * 0", differential: "0 4 * * 3" }
```

- Each repo is **one of** `volume` | `s3` | `gcs` | `azure`, by convention, not by schema
- Schedules are per repo, but **no `CronJob` is ever created**: the operator runs its own cron
- Retention is **not** a CRD field, it is a `global` passthrough to `pgbackrest.conf`

One backup, one object

```yaml
kind: PerconaPGBackup
spec:
  pgCluster: cluster1
  repoName: repo2
  options:
    - --type=full
```

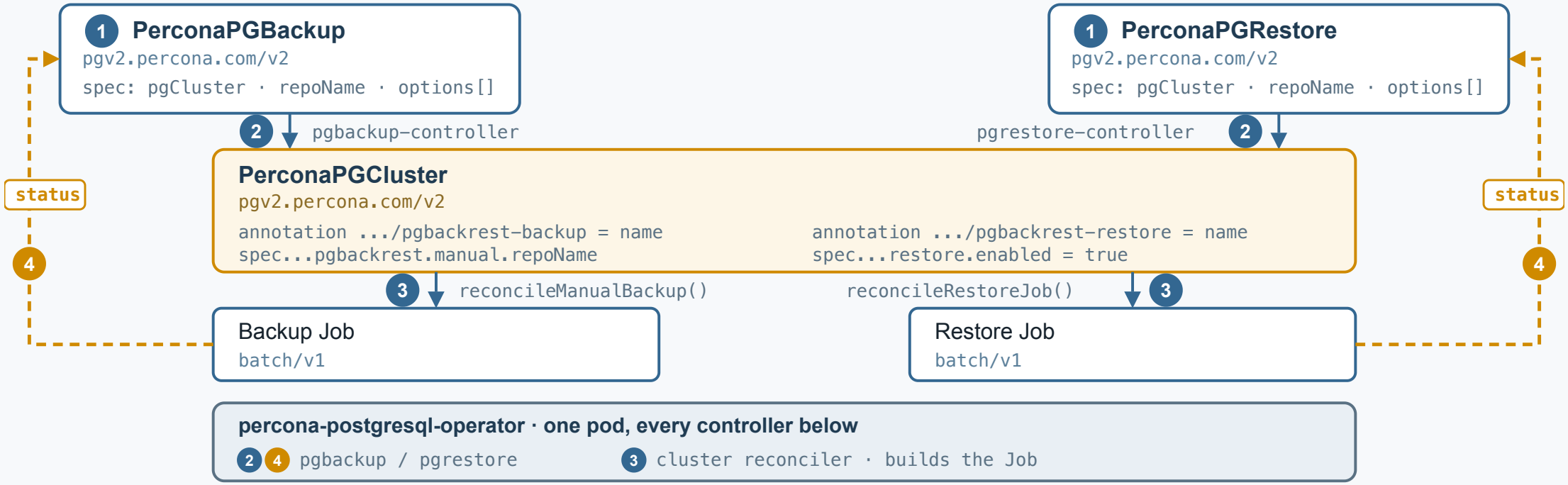
```yaml
kind: PerconaPGRestore
spec:
  pgCluster: cluster1
  repoName: repo2
  options:
    - --type=time
    - --target="2026-10-18 14:30:00"
```

`PerconaPGBackup.status`	what it gives you
`state`	`Starting` → `Running` → `Succeeded` | `Failed`
`backupType` / `storageType`	`full | differential | incremental` · `filesystem | s3 | gcs | azure`
`backupName` / `destination`	the backup's ID, `20261022-225152F`, and where it went, `s3://…/repo1`
`latestRestorableTime`	newest point you could recover to. **now minus this = your RPO**
`conditions[BackupLeaseAcquired]`	whether this backup is running or queued behind another

> `PerconaPGRestore.status` has only `jobName`, `state`, `completed`. Nothing about why.

PERCONA

What actually creates the Job



1 apply a CR · **2** write onto `PerconaPGCluster` · **3** create the Job · **4** copy status back

THE STORAGE

Four repos is not four copies of a backup

repo1 · S3

```
$ pgbackrest info --stanza=db --repo=1
stanza: db
  status: ok
  db (current)
    wal archive min/max (18):
      ...007 / ...00C
    full backup: 20261022-222858F
      database size: 23MB
      repo1: backup size: 3MB
    full backup: 20261022-223113F
      database size: 30.6MB
      repo1: backup size: 3.9MB
```

repo3 · Azure

```
$ pgbackrest info --stanza=db --repo=3
stanza: db
  status: error (no valid backups)
  db (current)
    wal archive min/max (18):
      ...005 / ...00C
```

Same cluster, seconds apart. **Identical WAL range** in both repos. Only `repo1` has backups.

Tiering four repos

Each repo buys a different recovery, not another copy

Repo	Where	Full backups	Keep	Why this rung
`repo1`	PVC, same AZ	weekly + daily incr	2 fulls	**fastest restore**
`repo2`	S3, same region	weekly + midweek diff	14 days	the everyday restore
`repo3`	another region or cloud	weekly	30 days	**survives losing a region or provider**
`repo4`	S3 infrequent-access	monthly	7 years	**compliance**, cheapest storage

- WAL goes to **all four, always**. Only the backup schedule differs, so PITR works from any of them
- Pick at restore time: `repo1` when someone dropped a table, `repo3` when the region is gone
- Cost lever: full backups are expensive, WAL is not. Vary the **full-backup frequency** per repo, never where WAL goes
- All four share **one `Lease` per cluster**, so backups never run in parallel: stagger the schedules

> `archive-push` **always** sends WAL to every repo. The operator turns on `archive-async: "y"` so one unreachable repo cannot stall the others to **one segment ahead** of it.

Bootstrap a new cluster from an existing repo

```
spec:
  dataSource:
    pgbackrest:
      stanza: db
      repo:
        name: repo1
        s3: { bucket: prod-pg-backups, endpoint: s3.eu-west-1.amazonaws.com, region: eu-west-1 }
      options: ["--type=time", "--target=2026-10-18 09:00:00"]
      configuration: [{ secret: { name: prod-backup-creds } }]
```

- Builds a **brand-new cluster** straight from a repo. The source cluster can be long gone: the repo outlives it
- The credentials only need **read** access to the bucket, so a team can clone prod without being able to touch prod's backups
- The same mechanism builds a **standby cluster** in another region: restore once, then replay WAL from the repo continuously

Where the abstraction leaks

- **A restore overwrites the cluster you point it at.** Its pods are torn down, `--delta` makes the data directory match the backup and deletes anything that is not in it, then the cluster rebuilds and replays WAL. No dry run, no rollback: for a copy, bootstrap a new cluster from the same repo
- **A backup object can vanish, and you never deleted it.** When retention expires a backup in the repo, the operator deletes the matching `PerconaPGBackup`. `kubectl get pg-backup` is a projection of `pgbackrest info`, not a record of what you ran
- **Nothing tests your restores.** `Succeeded` means the backup command exited zero, never that the data reads back. Green for a year proves nothing
- **The CRD is narrower and looser than the engine.** `^repo[1-4]$` caps you at four repos where pgBackRest allows 256; `global` is an unvalidated string map, so a typo'd key is ignored in silence

> Backups are a solved problem. Restores are an operational problem, and that part is still yours.

Three things to take away

1. The CR asks for a backup. pgBackRest and the repo hold the truth about whether you have one
2. Every repo gets the WAL stream, but a backup lands in **one** repo at a time, and that gap is where your tiering comes from
3. Pick a repo by **how long a restore from it takes**: a local PVC is fastest, another cloud is slowest but survives the most. Know the defaults the operator already chose: `archive-async` on, backups serialised by a lease, and WAL encryption changes both

Thank you

INTERESTING READS

Migrating off Crunchy

Hard fork of Crunchy's. The CRDs coexist during the move.

percona.com/postgresql/migrate-from-crunchy

Feature matrix

Everything the operator supports, version 3.1.0.

docs.percona.com/.../3.1.0/features.html

FIND ME

George Kechagias

Senior Software Engineer, Percona

linkedin.com/in/george-kechagias-86118b92

PERCONA®